01_视频内容介绍

视频目的：

◆总结自己的经验，回馈互联网

◆从零开始能独立编程

◆授人鱼不如授人以渔，掌握学习的能力

C/C++编程视频：

◆搭建自己的开发环境

◆C/C++的基本语法

◆动态库和应用程序编程

◆操作系统概念的编程

◆应用实例，日志系统编程

◆QT编程入门

02_编程环境介绍

宿主机:

◆VMware------安装虚拟机

◆SVN-----服务端搭建版本仓库

虚拟机1（Ubuntu）：

◆GCC-----C/C++编译调试环境

◆ftp-----服务器端，方便从别的机器上传代码编译

虚拟机2（Windows）：

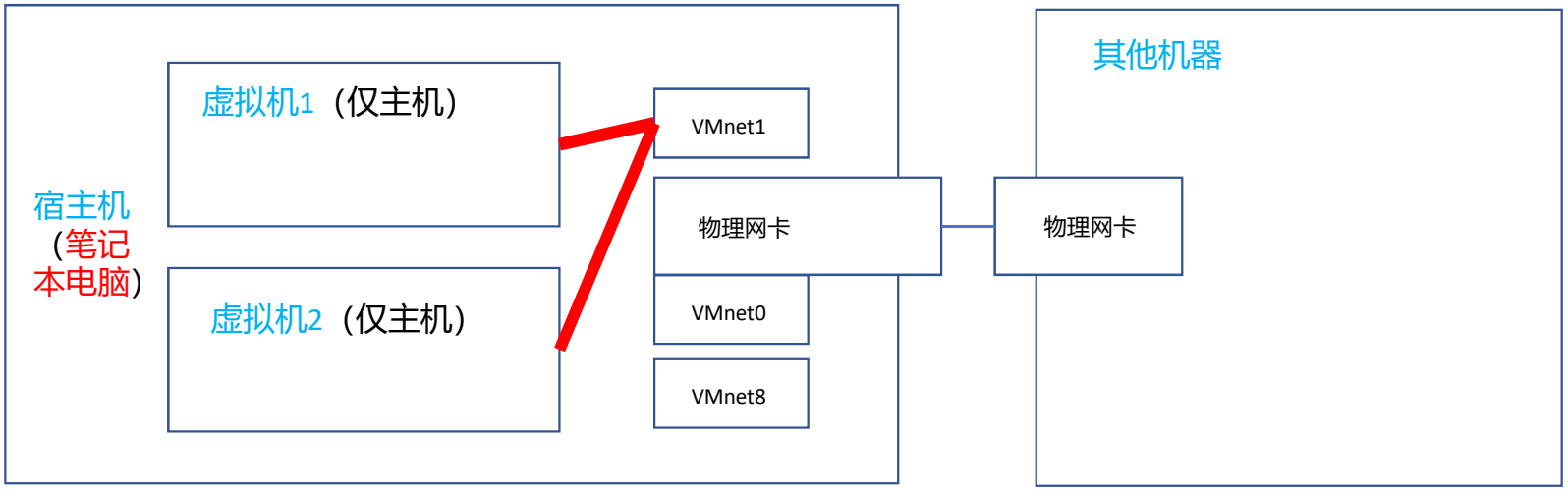
◆VS2019-----C/C++编译调试环境，svn，番茄助手，Qt插件

◆SVN-----客户端，代码版本管理

◆Xmanager-----使用ftp，ssh远程登录Linux机器的工具

03_VMware安装与配置 ①

向VMware致敬！

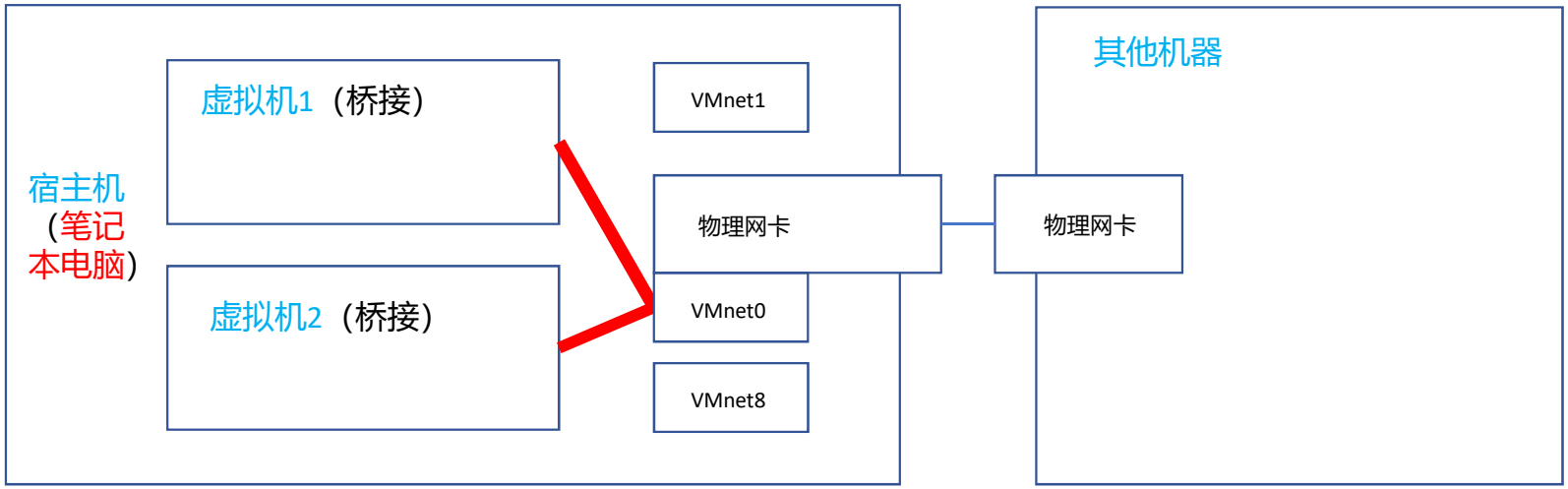


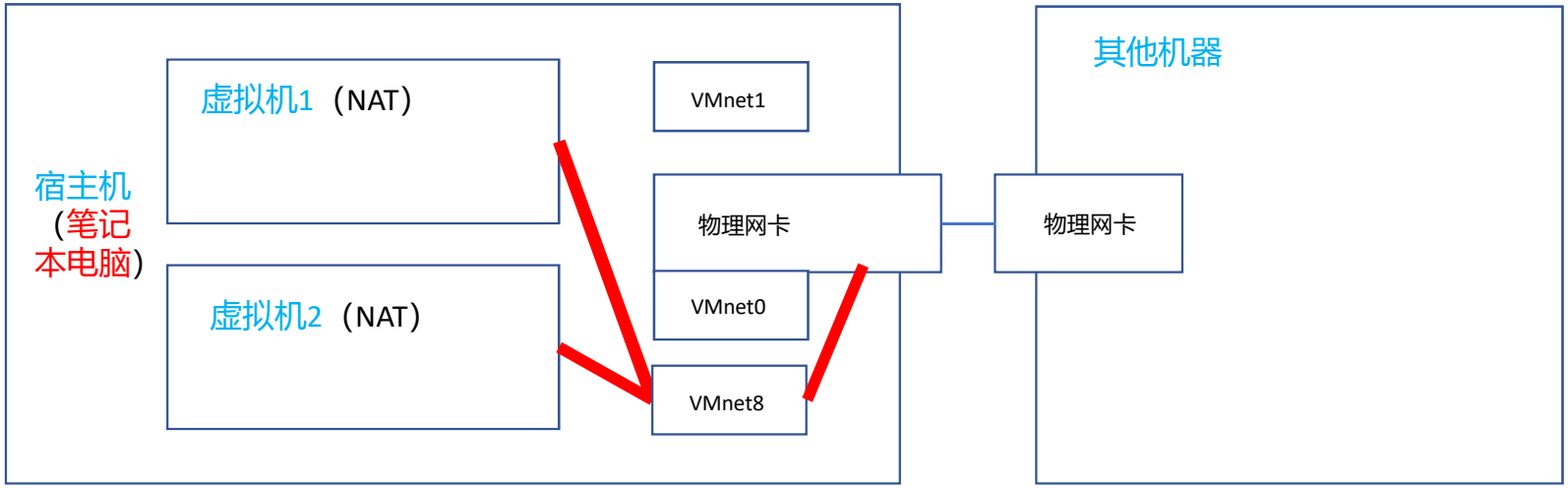
虚拟机的网络：

◆虚拟网卡：VMnet1、VMnet8、 VMnet0

◆三种网络模式：仅主机、NAT 、桥接

03_VMware安装与配置 ②



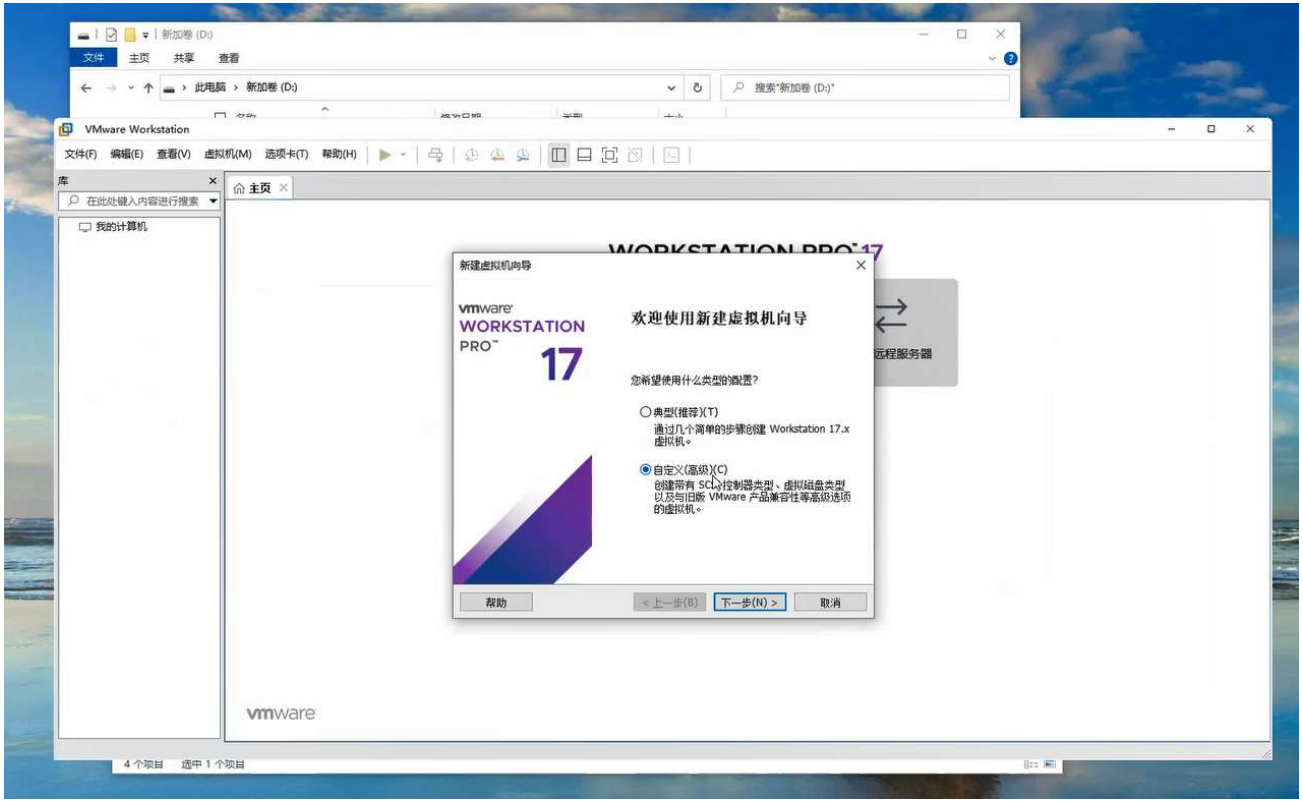


03_VMware安装与配置 ③

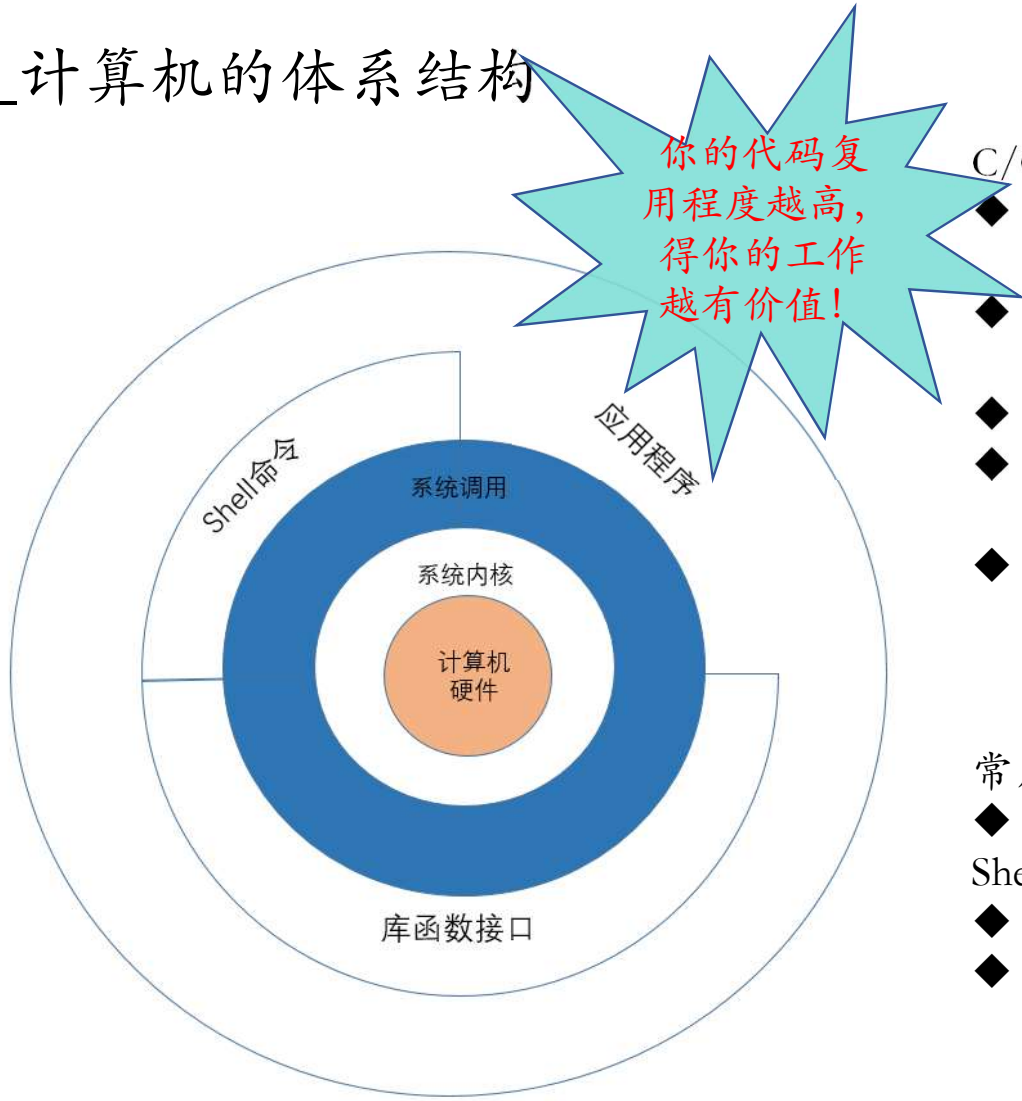
```
ubuntu@ubuntu-VMware:~/桌面$ ip route
default via 192.168.66.1 dev ens33 proto static metric 20100
192.168.66.0/24 dev ens33 proto kernel scope link src 192.168.66.212 metric 100
ubuntu@ubuntu-VMware:~/桌面$ ip neigh show
192.168.66.1 dev ens33 INCOMPLETE
192.168.66.100 dev ens33 lladdr 00:50:56:c0:00:01 REACHABLE
ubuntu@ubuntu-VMware:~/桌面$ ip route
default via 192.168.66.1 dev ens33 proto static metric 100
192.168.66.0/24 dev ens33 proto kernel scope link src 192.168.66.212 metric 100
ubuntu@ubuntu-VMware:~/桌面$ ip neigh show
192.168.66.1 dev ens33 lladdr 00:0c:29:46:c4:76 REACHABLE
192.168.66.100 dev ens33 lladdr 00:50:56:c0:00:01 REACHABLE
```

03_VMware安装与配置 ④

◆ 虚拟机的创建



04_计算机的体系结构



C/C++程序员的层次：

- ◆ 操作系统编写。Linus以及其他一些内核的维护者。
- ◆ 各种定制版Linux系统的开发者，嵌入式软件底层的开发。
- ◆ 硬件驱动的开发。NVIDIA的CUDA框架等
- ◆ 各种库，或者计算机工具的开发。Boost、SQLite，QT， OpenGL等。
- ◆ 应用程序的开发。比如我，比如很多同学的毕业设计（用C/C++写图书管理系统）等。

常用的shell命令：

- ◆ ls（dir）、cd、pwd、whereis、ls -l

Shell命令的批量执行：

- ◆ Linux下的sh文件。
- ◆ Windows下的bat文件。

05_Windows操作系统的安装

1、为什么要动手安装操作系统?

2、理解计算机的启动过程

3、根据提示安装操作系统

4、 系统激活

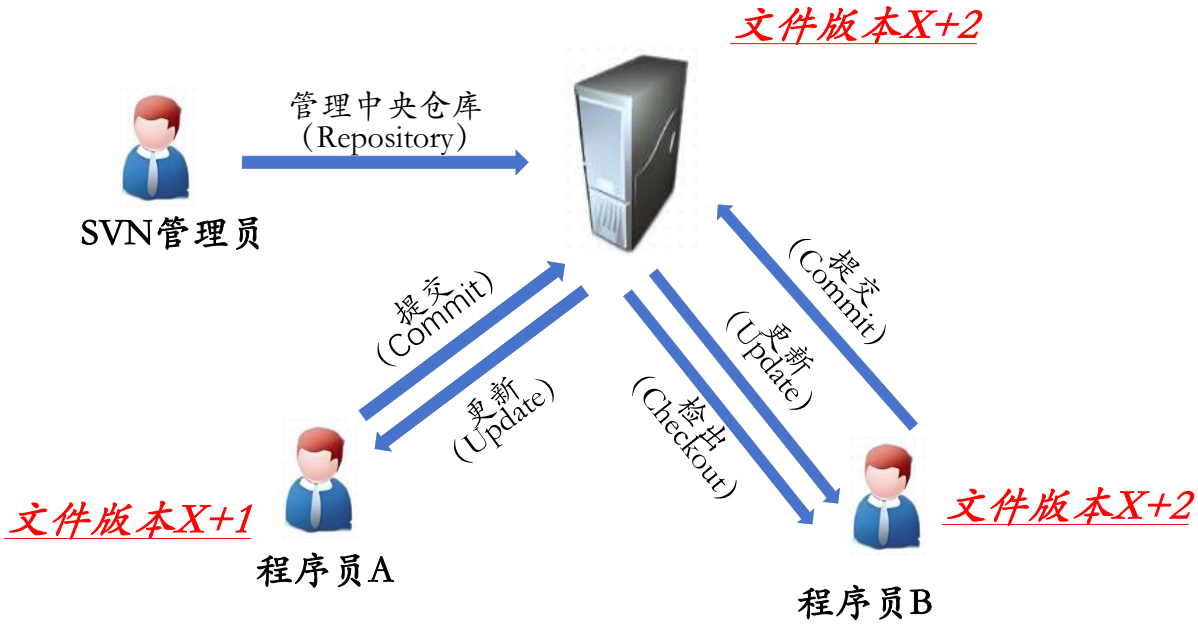
5、更新Windows的版本

6、Windows的设置与清理
- ◆ 根据喜好设置桌面
- ◆ 配置IP地址
- ◆ 更改计算机名称
- ◆ 设置远程访问

7、Windows环境变量的设置

8、虚拟机的相关设置
- ◆ 设置共享文件夹
- ◆ 快照标记

06_SVN的安装配置与使用 1



SVN服务端：

SVN服务端的功能包括存储版本库，管理权限和访问。Windows上可以通过VisualSVN Server提供的配置界面对SVN仓库进行管理。

SVN客户端：

SVN客户端用于与SVN服务端交互的工具，支持代码拉取、提交、合并冲突等操作。

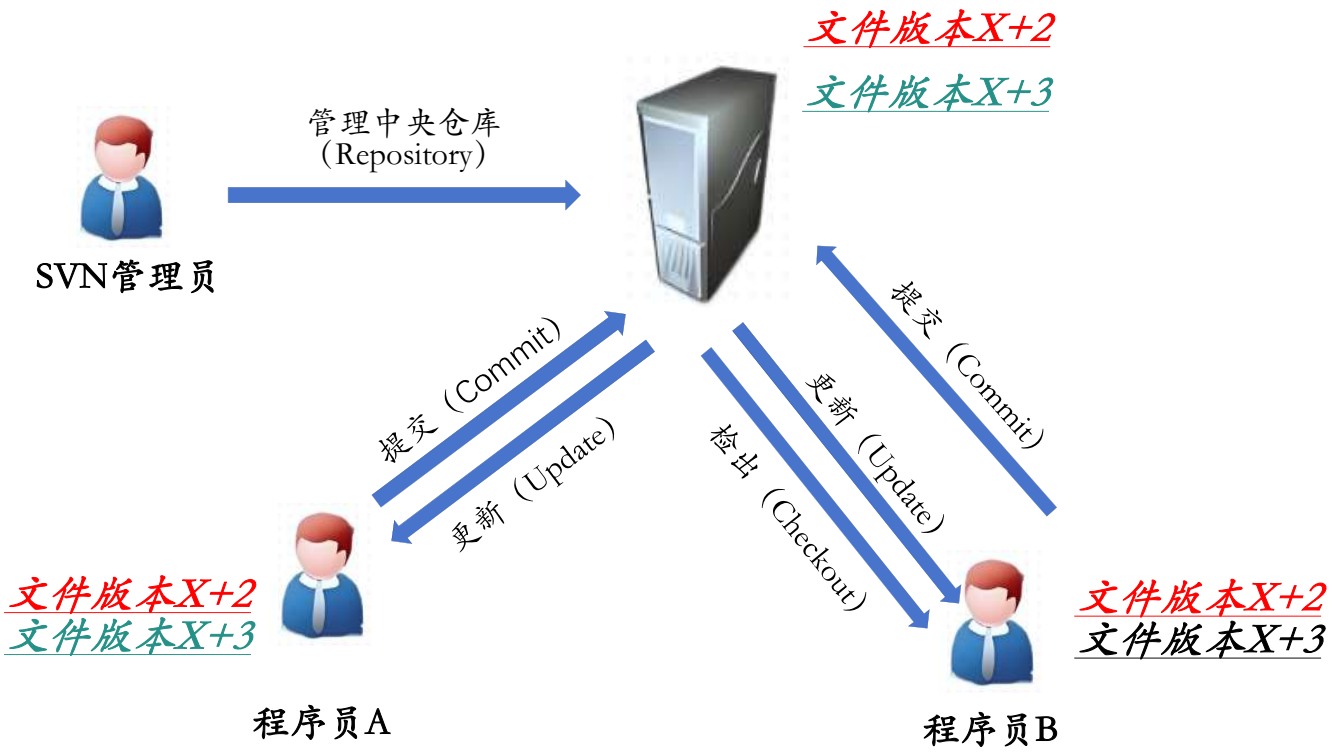
SVN用户：

SVN管理员、SVN用户。

SVN常用操作：

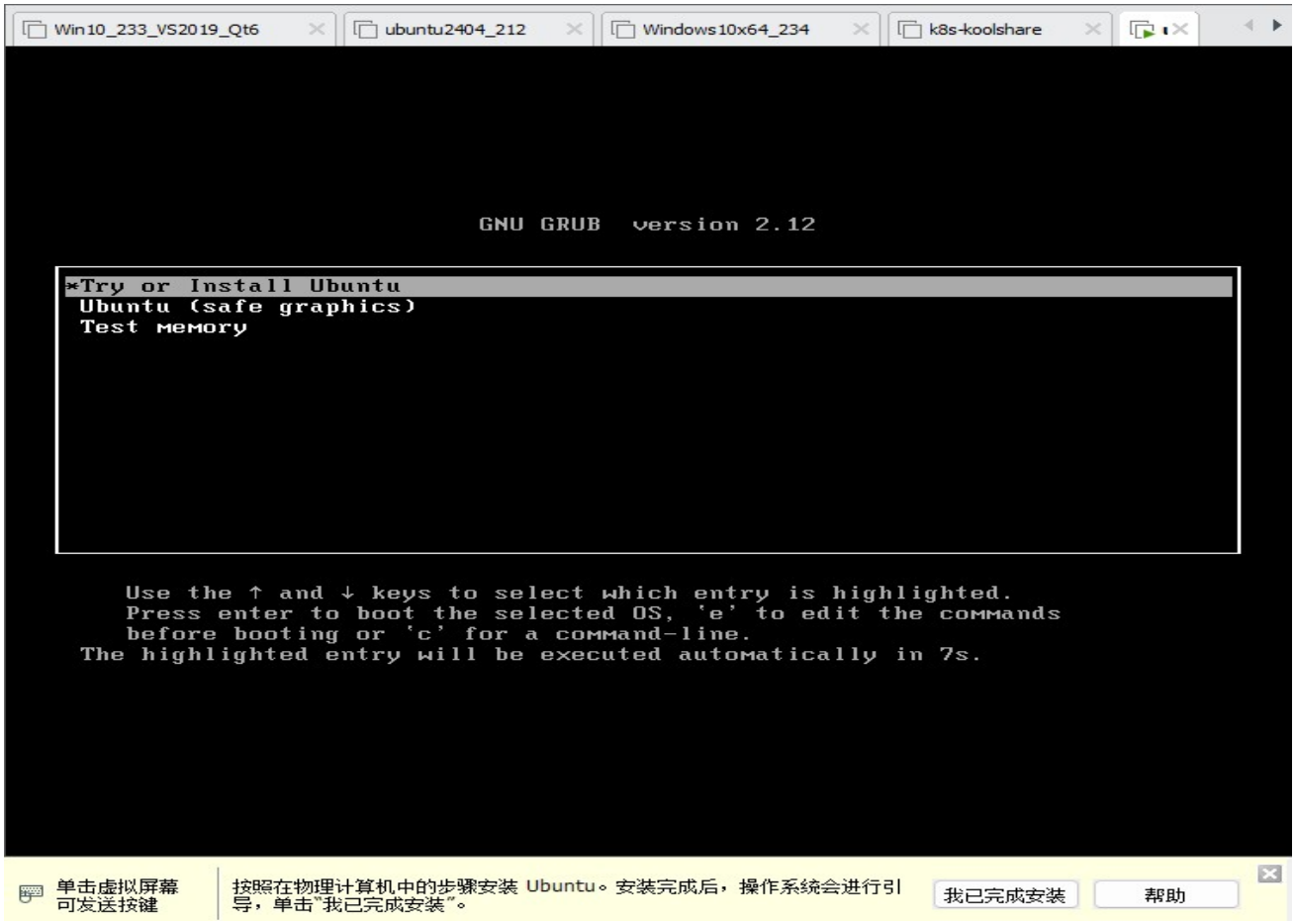
更新（Update）、提交（Commit）、检出（Checkout）。

06_SVN的安装配置与使用 ②



07_VS2019安装与配置

08_Ubuntu操作系统的安装



09_C++概述与学习途径 1

1、入门书籍

- 《C++ Primer》
- 《VC++ 深入详解》
- 《征服C/C++企业软件开发核心技术》

2、官方资料

- 微软的官方网站：
  https://learn.microsoft.com/zh-cn/
- 两个C/C++网站：
  https://cplusplus.com/
  https://zh.cppreference.com
- Linux下man命令
- Deep seek

3、学习过程阶段

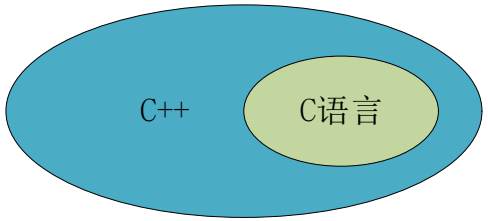
网上有个段子对C/C++的学习过程阶段的描述挺形象的：

- 1个月：都说C++难，但聪明如我，一个月就读完了C++primer，我看这C++也不过如此
- 3个月：原来之前一直在用C++语法写C...开始正式学习C++...
- 1年：今天花一下午一定要琢磨明白这段代码啥意思...
- 2年：C++太他妈难了，发明C++的可以吃X去了
- 3年：这辈子绝对不能继续搞C++了，珍惜生命，远离C++
- 4年：...
- 5年：终于神功初成，大家好，我是C++专家，疑难杂症请问我
- 7年：我比较擅长XXX这块，其他方面我是外行
- 10年：我有一定的C++基础...

09_C++概述与学习途径②

C/C++语言的关系

C++是C语言的一个超集，C++在C语言的基础上增加了面向对象，泛型等一些新的机制



如何学习C/C++

- 基础知识
- 计算机资源控制
- 高级编程

C/C++语言学什么

- C/C++的基本语法
- C/C++语言标准库
- C/C++语言标准规范在不断的变化

10_C++语法之main函数 1

不带参数的**main**函数

10_C++语法之main函数 ②

Coliru

coliru.stacked-crooked.com

Donate　　Coliru　　Restore defaults　Editor　Command　Help　Q&A

```cpp
#include <iostream>

void aftermain()
{
    std::cout << "call aftermain()\n";
}

class CBeforeMain
{
public:
    CBeforeMain()
    {
        std::cout << "call CBeforeMain()\n";
    }
};

CBeforeMain beforeMain;

int main(int argc, char* argv[])
{
    atexit(aftermain);
    std::cout << "Hello World!\n";
    std::cout << "argc: " << argc << std::endl;
    for (int i = 0; i < argc; i++)
    {
        std::cout << "argv["<<i<<"]:"<< argv[i] << std::endl;
    }
    return 0;
}
```

```
call CBeforeMain()
Hello World!
argc: 3
argv[0]:./a.out
argv[1]:22
argv[2]:hello
call aftermain()
```

```
g++ -std=c++20 -O2 -Wall -pedantic -pthread main.cpp && ./a.out 22 hello
```

Compile, link and run...　Share!

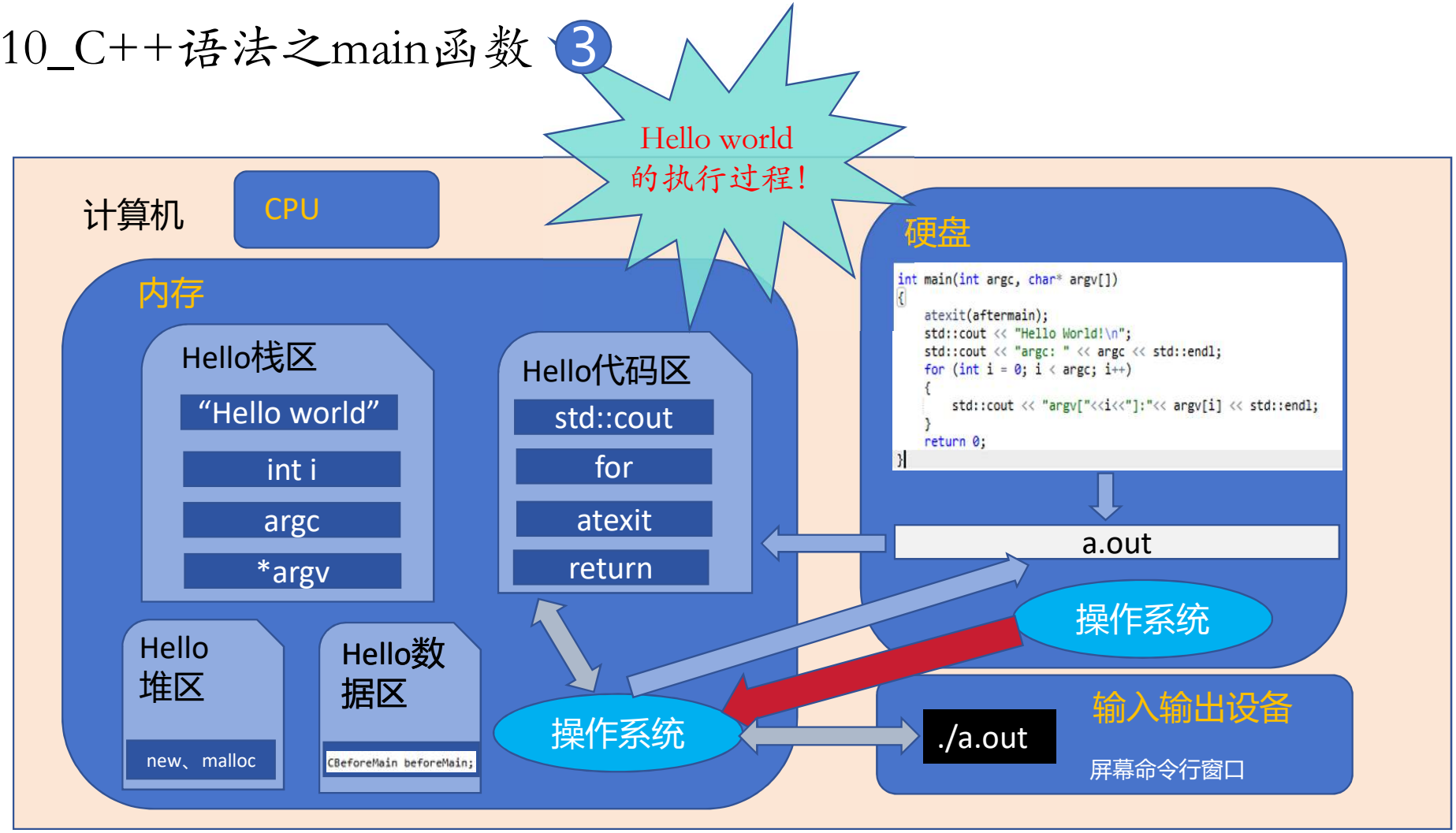
带参数的main函数

- argc、argv

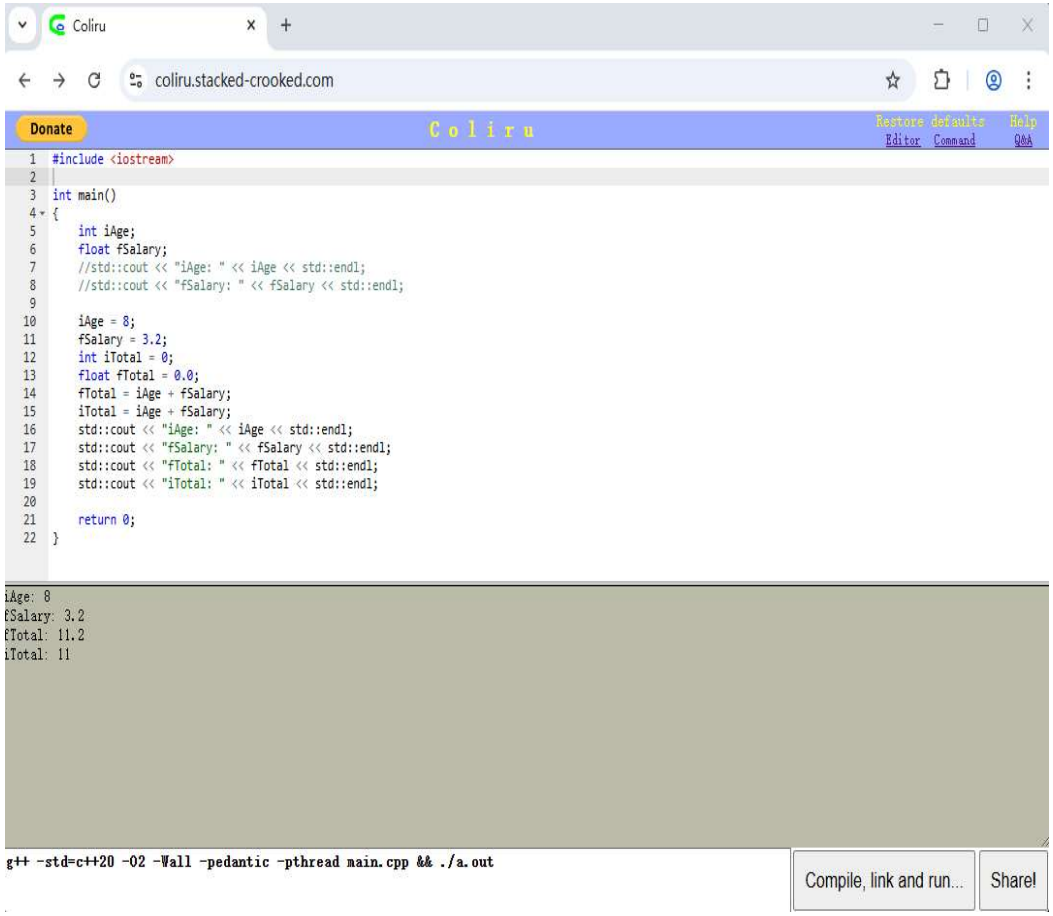
特殊函数

- atexit（）

10_C++语法之main函数 3



11_C++语法之变量 1



1、变量是什么

- ✓ 变量是内存中的命名空间

2、良好的编程习惯

- ✓ 变量命名要规范
- ✓ 变量要明确初始化

3、变量的隐式转换

- ✓ 基本变量的隐式转换
- ✓ 类对象赋值的隐式转换

11_C++语法之变量 ②

Coliru

coliru.stacked-crooked.com

Donate　Coliru　Restore defaults　Help　Editor　Command　Q&A

```cpp
#include <iostream>

#include <cstdint>

int main()
{
    int8_t iInt8;
    uint8_t uiInt8;
    int16_t iInt16;
    uint16_t uiInt16;
    int32_t iInt32;
    uint32_t uiInt32;
    int64_t iInt64;
    uint64_t uiInt64;
    int iInt;
    float fFloat;

    std::cout << "sizeof (iInt8) :" << sizeof(iInt8) << std::endl;
    std::cout << "sizeof (uiInt8) :" << sizeof(uiInt8) << std::endl;
    std::cout << "sizeof (iInt16) :" << sizeof(iInt16) << std::endl;
    std::cout << "sizeof (uiInt16) :" << sizeof(uiInt16) << std::endl;
    std::cout << "sizeof (iInt32) :" << sizeof(iInt32) << std::endl;
    std::cout << "sizeof (uiInt32) :" << sizeof(uiInt32) << std::endl;
    std::cout << "sizeof (iInt64) :" << sizeof(iInt64) << std::endl;
    std::cout << "sizeof (uiInt64) :" << sizeof(uiInt64) << std::endl;

    std::cout << "sizeof (int) :" << sizeof(iInt) << std::endl;
    std::cout << "sizeof (float) :" << sizeof(fFloat) << std::endl;

    return 0;
}
```

```
sizeof (iInt8) :1
sizeof (uiInt8) :1
sizeof (iInt16) :2
sizeof (uiInt16) :2
sizeof (iInt32) :4
sizeof (uiInt32) :4
sizeof (iInt64) :8
sizeof (uiInt64) :8
sizeof (int) :4
sizeof (float) :4
```

```
g++ -std=c++20 -O2 -Wall -pedantic -pthread main.cpp && ./a.out
```

Compile, link and run...　Share!

1、常用数值变量

- ✓ 整数类型
- ✓ 浮点数类型

2、整数变量

- ✓ int8_t、int16_t、int32_t、int64_t
- ✓ uint8_t、uint16_t、uint32_t、uint64_t
- ✓ 整数变量占用内存空间大小

3、浮点数变量

- ✓ float、double
- ✓ 浮点数变量占用内存空间大小

11_C++语法之变量 3

- 布尔变量

常常用整数常量1和0来表示true和false

- 字符变量

字符处理在C/C++中是比较复杂的，涉及到中文，英文，其他一些语言文字的各种编码，比较复杂

代码中一般要求除了注释之外，都不要使用中文

- 结构体

C语言可以利用结构体来实现类似C++类的功能，用struct关键字来定义一个结构体

字节对齐的概念要理解，保证指针操作时不出错

- 类对象

C++ 类（class）是面向对象编程（OOP）的核心，用于封装数据和行为，支持继承、多态和抽象等特性。用class关键字来实现一个类的定义。

sizeof计算一个类的对象的大小。

11_C++语法之变量 4

Coliru

coliru.stacked-crooked.com

Donate

Coliru

Restore defaults Editor Command

Help Q&A

```cpp
#include <iostream>

#include <cstdint>

typedef struct {
    char    cName[5];
    uint8_t uiAge;
    float   fHeignt;
} SStudent;

typedef struct {
    char     cChar;
    uint32_t uiInt32;
} SInt32AndChar;

typedef struct {
    char     cChar;
    uint16_t uiInt16;
} SInt16AndChar;

int main()
{
    SStudent struStudent;
    std::cout << "sizeof (SStudent) :" << sizeof(struStudent) << std::endl;

    SInt32AndChar struInt32AndChar;
    std::cout << "sizeof (SInt32AndChar) :" << sizeof(struInt32AndChar) << std::endl;

    SInt16AndChar struInt16AndChar;
    std::cout << "sizeof (SInt16AndChar) :" << sizeof(struInt16AndChar) << std::endl;

    return 0;
}
```

```
sizeof (SStudent) :12
sizeof (SInt32AndChar) :8
sizeof (SInt16AndChar) :4
```

```
g++ -std=c++20 -O2 -Wall -pedantic -pthread main.cpp && ./a.out
```

Compile, link and run... Share!

struct结构体

- ✓ C语言可以利用结构体来实现类似C++类的功能
- ✓ 字节对齐的概念

11_C++语法之变量 5

Coliru

不安全 https://coliru.stacked-crooked.com

Donate　　Coliru　　Restore defaults　Help　Editor　Command　Q&A

```cpp
#include <iostream>

#include <cstdint>

class CStudent {
public:
    // 构造函数（初始化成员变量）
    CStudent(std::string sName, int uiAge, float fHeignt)
        : m_sName(sName), m_uiAge(uiAge), m_fHeignt(fHeignt) {}

    // 成员函数
    void printInfo();

private:
    std::string m_sName;
    uint8_t m_uiAge;
    float   m_fHeignt;;
};

void CStudent::printInfo()
{
    std::cout << "sizeof (m_sName) :" << sizeof(m_sName) << std::endl;
    std::cout << "sizeof (m_uiAge) :" << sizeof(m_uiAge) << std::endl;
    std::cout << "sizeof (m_fHeignt) :" << sizeof(m_fHeignt) << std::endl;
}

int main()
{
    CStudent student1("ZhangSan",12,163.5);
    student1.printInfo();
    std::cout << "sizeof (student1) :" << sizeof(student1) << std::endl;

    return 0;
}
```

```
sizeof (m_sName) :32
sizeof (m_uiAge) :1
sizeof (m_fHeignt) :4
sizeof (student1) :40
```

```
g++ -std=c++20 -O2 -Wall -pedantic -pthread main.cpp && ./a.out
```

Compile, link and run...　Share!

Microsoft Visual Studio 调试控制台

```
sizeof (m_sName) :40
sizeof (m_uiAge) :1
sizeof (m_fHeignt) :4
sizeof (student1) :48

E:\crossplatv2\x64\Debug\chapter01.exe (进程 10060)已退出，代码为 0。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口. . .
```

C++类

- ✓ 面向对象编程思想
- ✓ 继承、多态、封装
- ✓ 类对象内存空间的大小
- ✓ sizeof计算类对象空间大小

12_C++语法之指针 1

Coliru

coliru.stacked-crooked.com

Donate　Coliru　Restore defaults　Editor　Command　Help　Q&A

```cpp
#include <iostream>

int add(int iAddNum1, int iAddNum2)
{
    iAddNum1 = iAddNum1 + 1;
    std::cout << "int add(int iAddNum1, int iAddNum2):  iAddNum1=" << iAddNum1 << std::endl;
    std::cout << "&iAddNum1:  " << &iAddNum1 << std::endl;
    return 1;
}

int add(int iAddNum1)
{
    iAddNum1 = iAddNum1 + 1;
    std::cout << "int add(int iAddNum1): iAddNum1= " << iAddNum1 << std::endl;
    std::cout << "&iAddNum1:  " << &iAddNum1 << std::endl;
    return 1;
}

//int add(int& iAddNum1)
//{
    //iAddNum1 = iAddNum1 + 1;
    //std::cout << "int add(int& iAddNum1): iAddNum1= " << iAddNum1 << std::endl;
```

```
&i1: 0x7ffc22e81d34
i3: 0
&i3: 0x7ffc22e81d38

1 before int add(i1, i2):  i1=0
int add(int iAddNum1, int iAddNum2):  iAddNum1=1
&iAddNum1:  0x7ffc22e81d0c
1 after int add(i1, i2): i1=0

2 before int add(i1): i1=0
int add(int iAddNum1): iAddNum1= 1
&iAddNum1:  0x7ffc22e81d0c
2 after int add(i1): i1=0

3 before int    add(&i1): i1=0
int add(int* iAddNum):  *iAddNum1=1
iAddNum1:  0x7ffc22e81d34
3 after int     add(&i1): i1=1

4 before int add(i3): i3=0
4 after int add(i3): i3=0
```

```
g++ -std=c++20 -O2 -Wall -pedantic -pthread main.cpp && ./a.out
```

Compile, link and run...　Share!

指针、地址、引用

- ✓ 地址是内存中每个字节的唯一标识符
- ✓ 指针存储的是内存地址
- ✓ 引用是变量的别名

12_C++语法之指针 ②

Coliru

coliru.stacked-crooked.com

Donate　　Coliru　　Restore defaults Editor Command　　Help Q&A

```cpp
#include <iostream>

int add(int iAddNum1, int iAddNum2)
{
    iAddNum1 = iAddNum1 + 1;
    std::cout << "int add(int iAddNum1, int iAddNum2):  iAddNum1=" << iAddNum1 << std::endl;
    std::cout << "&iAddNum1:  " << &iAddNum1 << std::endl;
    return 1;
}

//int add(int iAddNum1)
//{
    //iAddNum1 = iAddNum1 + 1;
    //std::cout << "int add(int iAddNum1): iAddNum1= " << iAddNum1 << std::endl;
    //std::cout << "&iAddNum1:  " << &iAddNum1 << std::endl;
    //return 1;
//}

int add(int& iAddNum1)
{
    iAddNum1 = iAddNum1 + 1;
    std::cout << "int add(int& iAddNum1): iAddNum1= " << iAddNum1 << std::endl;
```

```
&i1: 0x7fff98a07b14
i3: 0
&i3: 0x7fff98a07b18

1 before int add(i1, i2):  i1=0
int add(int iAddNum1, int iAddNum2):  iAddNum1=1
&iAddNum1:  0x7fff98a07aec
1 after int add(i1, i2): i1=0

2 before int add(i1): i1=0
int add(int& iAddNum1): iAddNum1= 1
&iAddNum1:  0x7fff98a07b14
2 after int add(i1): i1=1

3 before int    add(&i1): i1=1
int add(int* iAddNum):  *iAddNum1=2
iAddNum1:  0x7fff98a07b14
3 after int     add(&i1): i1=2

4 before int add(i3): i3=0
4 after int add(i3): i3=0
```

```
g++ -std=c++20 -O2 -Wall -pedantic -pthread main.cpp && ./a.out
```

Compile, link and run...　　Share!

指针、引用作为函数参数的区别

12_C++语法之指针 3

Coliru

coliru.stacked-crooked.com

Donate　Coliru　Restore defaults　Help　Editor　Command　Q&A

```cpp
#include <iostream>

int iArray[5] =
{
    0,1,2,3,4
};

int main()
{
    int* iP01 = &iArray[0];
    std::cout << "iP01=" << iP01 << " *iP01=" << *iP01 << std::endl;
    int* iP02 = iArray;
    std::cout << "iP02=" << iP02 << " *iP02=" << *iP02 << std::endl;
    std::cout << std::endl;

    int* iP31 = &iArray[3];
    std::cout << "iP31=" << iP31 << " *iP31=" << *iP31 << std::endl;
    int* iP32 = iArray+3;
    std::cout << "iP32=" << iP32 << " *iP32=" << *iP32 << std::endl;
}
```

```
iP01=0x601290 *iP01=0
iP02=0x601290 *iP02=0

iP31=0x60129c *iP31=3
iP32=0x60129c *iP32=3
```

```
g++ -std=c++20 -O2 -Wall -pedantic -pthread main.cpp && ./a.out
```

Compile, link and run...　Share!

指针和数组

- ✓ 指针和数组的关系
- ✓ 指针的加法

12_C++语法之指针 4

Coliru

coliru.stacked-crooked.com

Donate

Coliru

Restore defaults Editor Command

Help Q&A

```cpp
#include <iostream>

int add1(int iNum)
{
    std::cout << "add1(int iNum) iNum=" << iNum << std::endl;
    iNum = iNum + 1;
    std::cout << "add1(int iNum) return:" << iNum << std::endl;
    return iNum;
}

int add2(int iNum)
{
    std::cout << "add2(int iNum) iNum=" << iNum << std::endl;
    iNum = iNum + 2;
    std::cout << "add2(int iNum) return:" << iNum << std::endl;
    return iNum;
}

int main()
{
    int  (*pFunc)(int i);

    int i = 0;
    pFunc = add1;
    (*pFunc)(i);
    pFunc = add2;
    (*pFunc)(i);
}
```

```
add1(int iNum) iNum=0
add1(int iNum) return:1
add2(int iNum) iNum=0
add2(int iNum) return:2
```

```
g++ -std=c++20 -O2 -Wall -pedantic -pthread main.cpp && ./a.out
```

Compile, link and run... Share!

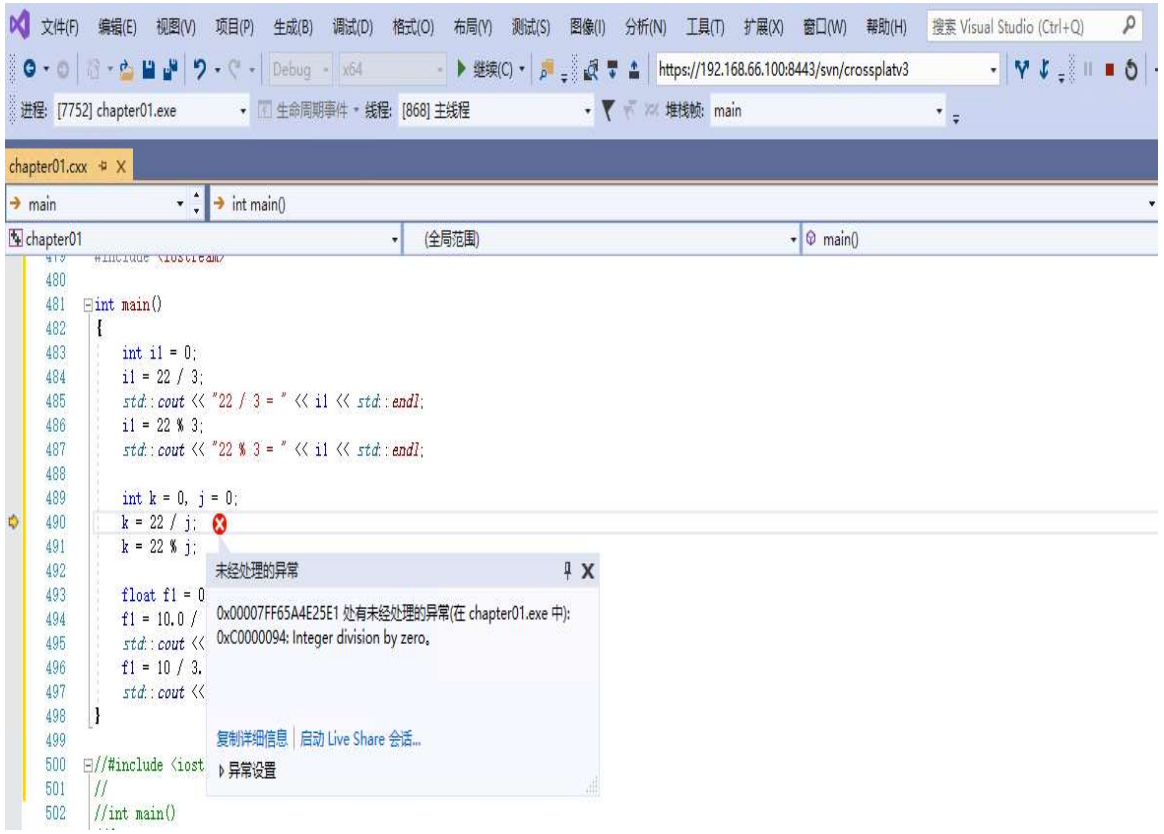
函数指针

- ✓ 指针指向一个函数

动态内存分配

- ✓ 标准库函数malloc和free
- ✓ 运算符new和delete

13_C++语法之运算符及其他 1



- **C/C++的除法**
  - ✓ 整数除法
  - ✓ 浮点数除法
  - ✓ 千万别用0作为除数
- **浮点数相等的判断**

13_C++语法之运算符及其他 ②

- 动态内存分配

malloc和free是C/C++语言中用于动态内存管理的重要函数，不是运算符而是标准库函数

new和delete是用于动态内存管理的运算符

- 关系运算

“>、<、>=、<=、==、!=”。注意浮点数比较时，不建议使用“0.0==a”这种写法，要从计算机计算的原理想想为什么

- 逻辑运算

”&&、||、！、&、|、^、~”

- 算术运算

“*、/、%、+、-”。要特别注意的是，“/”和“%”不要除以0，空指针和除数为0是C/C++最大的噩梦，直接导致程序退出

“+、-”有两种用法，比如“i=i+”，“i=i+2”

- 赋值运算

”=、+=、-=、*=、/=、%=、<<=、>>=、&=、|=”

- 其他运算符

位移运算符：“>>、<<”；
自增和自减：“++、--”；
条件运算：“？：”；
逗号运算：“，”。

13_C++语法之运算符及其他 ③

C/C++支持顺序、分支、循环等控制语句，用到的一些关键如下：

- 分支语句，if…else和switch；
- 循环语句，while、do…while和for；
- 循环的控制，break和continue；
- 跳转语句，goto（这个一般公司都禁用）

13_C++语法之运算符及其他 4

面向对象编程、其他语言特征！

- 面向对象编程

✓ 封装、继承、多态

✓ static关键字修饰类的成员函数和成员变量的功能

- 类模板

C/C++标准库中的STL标准模板库从C++11开始才基本成熟，这之前的代码库中可能会有类模板这些写法。

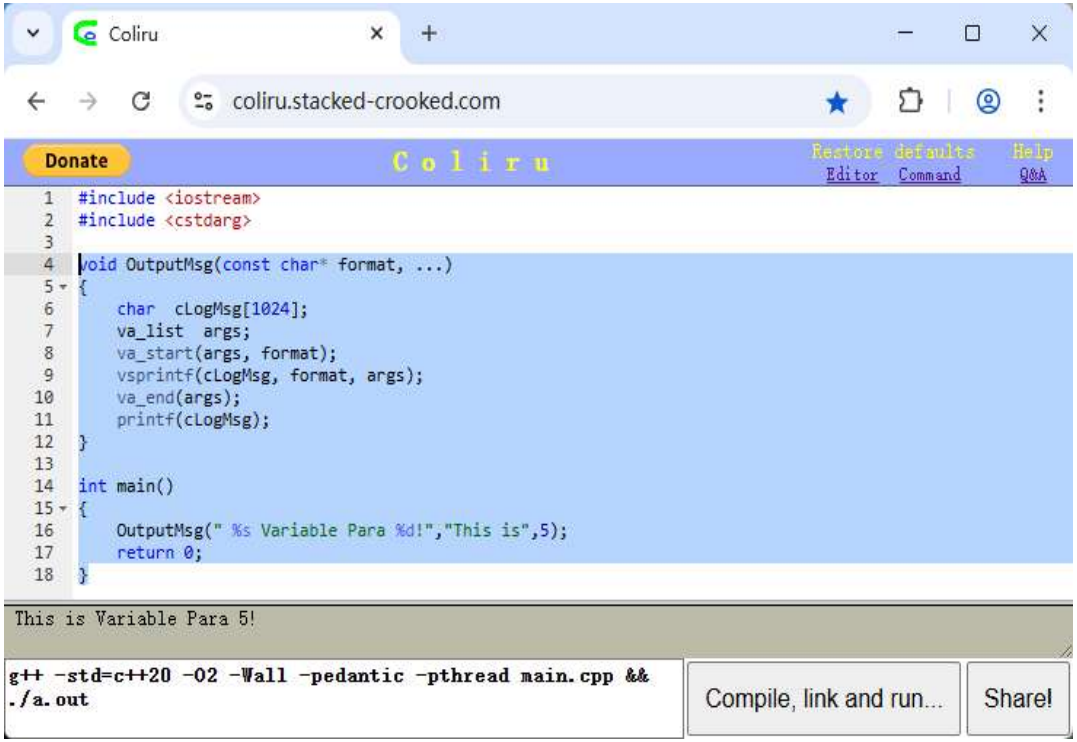
- 可变参数函数

可变参数在编程中经常用到，可以参看实例体会下

- 正则表达式和Lambda表达式

需要时学习，优点是简洁，语义强大

13_C++语法之运算符及其他 5



- 可变参数函数编程

13_C++语法之运算符及其他 ⑥

1、C/C++基本语法----main函数

2、C/C++基本语法----hello word的执行

3、C/C++基本语法----变量

4、C/C++基本语法----指针

5、C/C++基本语法----运算符

6、C/C++基本语法----流程控制

7、C/C++基本语法----面向对象编程、其他

C/C++语法总结！

- 1.4.3 C/C++变量
  - 1.4.3.1 数值变量
  - 1.4.3.2 布尔变量
  - 1.4.3.3 字符变量
  - 1.4.3.4 结构体
  - 1.4.3.5 类对象
- 1.4.4 C/C++指针
  - 1.4.4.1 指针、地址和引用
  - 1.4.4.2 指针和数组
  - 1.4.4.3 函数指针
  - 1.4.4.4 动态内存分配
- 1.4.5 C/C++运算符
  - 1.4.5.1 算术运算
  - 1.4.5.2 关系运算
  - 1.4.5.3 逻辑运算
  - 1.4.5.4 赋值运算
  - 1.4.5.5 其他运算符
- 1.4.6 C/C++流程控制
- 1.4.7 面向对象编程
  - 1.4.7.1 封装与继承
  - 1.4.7.2 多态
- 1.4.8 C/C++其他语言特性
  - 1.4.8.1 可变参数函数
  - 1.4.8.2 类模板
  - 1.4.8.3 正则表达式和Lambda表达式

14_创建应用程序与动态库

大纲：

1. 创建一个 exam_win 目录，在这个目录下：

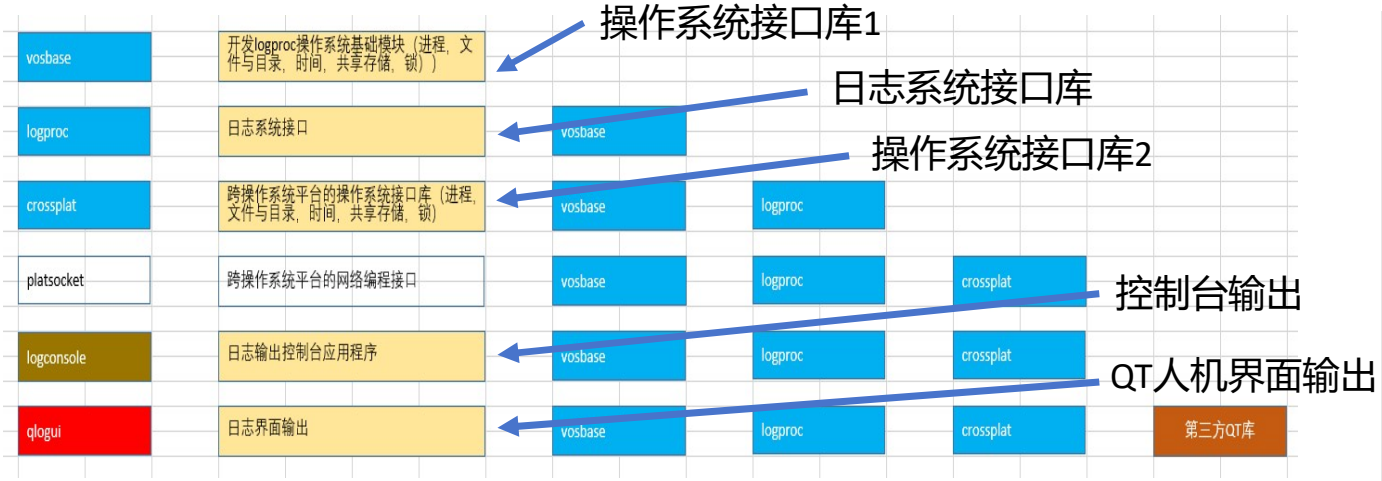
- 创建一个 VS2019 带 hello 的控制台应用程序 example1
- 创建一个 VS2019 带 hello 的动态库 example2
- example1 调用 example2 模块
- 单步执行调试控制台应用程序
- 单步执行调试动态库程序

15_跨平台的应用程序与动态库

2. 创建一个 exam_cross 目录，在这个目录下：

- example1 和 example2 上传到 SVN，下载下来，移植性存在很多问题，改进。
- 创建一个 VS2019 空的控制台应用程序 example3
- 创建一个 VS2019 空的动态库 example4
- example3 调用 example4 模块
- 设置环境变量，调整目录结构
- 编写 3 个 Makefile 文件，介绍 Makefile 文件和 Windows 下项目工程文件作用
- 上传代码到 Linux 机器，在 Linux 下编译和运行两个示例

16_高级编程构思与项目设计



- 1 Qt简介
  - 1.1 实践：Qt网站浏览介绍
  - 1.2 Qt是什么
  - 1.3 关于Qt的版权
  - 1.4 Qt Framework
  - 1.5 Qt Creator
  - 1.6 Qt Designer
  - 1.7 Qt Linguist
  - 1.8 Qt VS 插件
- 2 Windows下Qt安装
- 3 Linux下Qt安装
- 4 Qt编程之helloword
  - 4.1 Qt项目的创建
  - 4.2 实践：简单界面环境的实现
  - 4.3 创建VS2019项目
  - 4.4 翻译文件的使用
  - 4.5 创建Linux项目
- 5 实例演示
  - 5.1 实践：Qt界面程序编程
  - 5.2 创建项目
  - 5.3 Qt Designer使用
  - 5.4 目录结构
  - 5.5 代码简单分析
    - 5.5.1 代码的一般技巧
    - 5.5.2 Qt的几个UI类
    - 5.5.3 几个非UI类
    - 5.5.4 qlogui的翻译文件
- 6 其他Qt类的简介
  - 6.1 多媒体
  - 6.2 数据库
  - 6.3 网络通讯
  - 6.4 多线程
  - 6.5 文件

- 1 进程与线程
  - 1.1 实践：进程与线程编程实践
  - 1.2 进程的理解
  - 1.3 创建和终止线程
  - 1.4 进程线程相关信息
- 2 文件与目录
  - 2.1 实践：文件与目录编程实践
  - 2.2 不同操作系统目录
  - 2.3 基本的操作接口
  - 2.4 为什么不使用Qt
- 3 时间
  - 3.1 实践：时间处理编程实践
  - 3.2 时间获取
  - 3.3 时间转换
  - 3.4 系统时间操控
- 4 共享存储
  - 4.1 实践：共享存储编程实践
  - 4.2 共享内存的创建
  - 4.3 查看系统共享内存
  - 4.4 进程与共享存储的理解

- 1 实践：创建日志应用系统
- 2 系统概念与需求
  - 2.1 系统概念
  - 2.2 分类记录日志
  - 2.3 日志输出
- 3 日志系统的使用
  - 3.1 简单的日志系统
  - 3.2 系统整体架构分层
  - 3.3 定制自己的日志系统
  - 3.4 日志系统自身日志输出
- 4 系统架构
  - 4.1 数据结构定义
  - 4.2 日志配置
  - 4.3 日志系统共享内存
  - 4.4 回调接口设计
  - 4.5 公共头文件
  - 4.6 日志输出设计
- 5 控制台输出进程
  - 5.1 入口参数
  - 5.2 回调函数实现
  - 5.3 控制台进程